

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Bash Shell and cron

What is a Shell?

A **shell** is the **Linux command-line interpreter**.

- It provides an **interface between the user and the kernel**
- It reads user commands and executes them
- Commands may be:
 - Built-in shell commands
 - External programs
 - Scripts
 - User-defined applications

Example

If the user types:

```
ls
```

The shell interprets the command and asks the kernel to execute the ls program.

Shell Command Syntax

Commands entered at the shell prompt follow this syntax:

```
command [arg1] [arg2] ... [argn]
```

- Items in [] are optional
- Commands may run **with or without arguments**

Command Processing

1. Shell reads the entire line after pressing **Enter**
2. Shell parses the input into:
 - Command name
 - Options
 - Filenames
3. Shell searches for the command:
 - In the specified path (e.g., ./mycommand)
 - Or in directories listed in the **\$PATH** environment variable
4. If syntax is correct, kernel executes the command

Various Shells

Linux supports multiple shells:

- **sh**
Bourne Shell (original UNIX shell)
- **csh, tcsh** - C-shell and its enhanced version
- **ksh**
Korn Shell
- **bash**
Bourne Again Shell
 - Most popular shell
 - Default in most Linux distributions
 - Developed by GNU

Command to check current shell

echo \$SHELL

Environment Variables

- Defined for the **current shell**
- Inherited by **child processes**
- Used to pass information to spawned programs

Shell Variables

- Exist **only in the current shell**
- Not inherited by child shells
- Used for temporary data (e.g., counters, paths)

Naming Convention

- Usually written in **uppercase letters**

Viewing Environment Variables

env

or

printenv

Common Environment Variables

- **SHELL**
Shell interpreting the commands (usually bash)
- **TERM**
Terminal type being emulated
- **USER**
Current logged-in user
- **PWD**
Present working directory

- **OLDPWD**
Previous working directory (cd - uses this)
- **PATH**
List of directories searched for commands
- **HOME**
User's home directory

Common Shell Variables

- **BASHOPTS**
Options enabled when bash was started
- **BASH_VERSION**
Bash version in human-readable form
- **BASH_VERSINFO**
Bash version in machine-readable form

Shell Basics

Creating a Shell Variable

varname=value

Accessing a Variable

echo \$varname

Child Shell Behavior

- When a new shell is spawned:
 - Shell variables from parent are **not available**

Exiting Child Shell

exit

Exporting Variables

To make a variable available to child shells:

export varname

Removing a Shell Variable

unset varname

Setting Environment Variables at Login

- Edit the .profile file in the **\$HOME** directory
- Add export commands

Example:

export PATH=\$PATH:/my/custom/path

This ensures variables are set automatically on login.

Shell Control Flow – if Statement

Shell scripts support conditional execution.

Example: Checking if a File Exists

```
if [ -f filename ]
then
    echo "File exists"
else
    echo "File does not exist"
fi
```

Concept to remember: if evaluates a condition and executes commands accordingly.

Shell Control Flow – Loops

Shell supports looping constructs such as:

- for
- while
- until

Nested Loop

- A loop inside another loop
- Used for multi-level iteration

Example:

```
#!/bin/bash

for i in {1..5}
do
    echo "i = $i"
done

#!/bin/bash

for i in {1..5}
do
    for j in {1..3}
    do
        echo "i = $i and j = $j"
    done
done
```

Shell Control Flow – break and continue

- **break**
 - Exits the loop immediately
- **continue**
 - Skips the current iteration
 - Moves to the next iteration

Used to control loop execution flow.

```
#!/bin/bash

for i in {1..5}
do
    if [ $i -eq 3 ]
    then
        continue
    fi

    for j in {1..3}
    do
        echo "i = $i and j = $j"
    done
done
```

```
#!/bin/bash

for i in {1..5}
do
    if [ $i -eq 3 ]
    then
        break
    fi

    for j in {1..3}
    do
        echo "i = $i and j = $j"
    done
done
```

cron

What is cron?

- **cron** is a **daemon** (background service)
- Runs continuously after system boot
- Executes scheduled tasks automatically

The cron daemon is called **crond**.

cron Functionality

- Cron reads instructions from **crontab** (cron tables)
- Each user can have their own crontab
- Jobs are scheduled using a **specific syntax**
- Used for:
 - Backups
 - Log cleanup
 - Periodic monitoring
 - Automation

Checking cron Jobs

List scheduled cron jobs

```
crontab -l
```

Managing cron Service

- Check cron status
- Start cron service if not running
- Cron runs automatically at system boot in most systems

Adding Jobs to cron

Edit crontab

```
crontab -e
```

This command allows:

- Adding
- Editing
- Deleting cron jobs

cron Syntax

A cron job follows this format:

minute hour day month weekday command

SCHEDULE	SCHEDULED VALUE
5 0 * 8 *	At 00:05 in August.
5 4 * * 6	At 04:05 on Saturday.
0 22 * * 1-5	At 22:00 on every day-of-week from Monday through Friday.

cron Examples

Example: Execute a Program Every Minute

* * * * * command

View cron jobs

crontab -l

```
# m h dom mon dow  command
*/1 * * * * /bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log
```

```
ubuntu@ip-172-31-41-208:~/OS$ tail /var/log/syslog
Jan 24 08:38:01 ip-172-31-41-208 CRON[2923]: (ubuntu) CMD ($HOME/OS/hello.exe)
Jan 24 08:38:01 ip-172-31-41-208 CRON[2920]: (CRON) info (No MTA installed, discarding output)
Jan 24 08:39:01 ip-172-31-41-208 CRON[2927]: (ubuntu) CMD ($HOME/OS/hello.exe)
Jan 24 08:39:01 ip-172-31-41-208 CRON[2926]: (CRON) info (No MTA installed, discarding output)
Jan 24 08:39:09 ip-172-31-41-208 crontab[2911]: (ubuntu) REPLACE (ubuntu)
Jan 24 08:39:09 ip-172-31-41-208 crontab[2911]: (ubuntu) END EDIT (ubuntu)
Jan 24 08:39:41 ip-172-31-41-208 crontab[2932]: (ubuntu) LIST (ubuntu)
Jan 24 08:40:01 ip-172-31-41-208 cron[665]: (ubuntu) RELOAD (crontabs/ubuntu)
Jan 24 08:40:01 ip-172-31-41-208 CRON[2935]: (ubuntu) CMD (/bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log)
Jan 24 08:41:01 ip-172-31-41-208 CRON[2939]: (ubuntu) CMD (/bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log)
```